

ML-Based LLVM Optimization Pass Ordering & Selection

Automated Static Feature Extraction, Limited-Budget Empirical Profiling,
Supervised Pass Sequence Prediction, and Empirical Evaluation against LLVM -O2
and -Oz

Project Domain:	Compiler Design & Machine Learning (LLVM)
Primary Metric:	Code Size (.text section bytes)
Toolchain:	LLVM 23.1.2 / Clang / llvm-size / Python 3.12
Implementation Status:	70% Milestone Fully Executed & Verified
Cloud CI/CD:	GitHub Actions (Passing / Green Checkmark)

+10.36%

AVERAGE CODE SIZE
REDUCTION VS -O2 / -
OZ

+45.60%

PEAK SINGLE PROGRAM
WIN (CONV2D)

100%

WIN / TIE RATE (ZERO
REGRESSIONS)

4.27%

AVERAGE DISTANCE TO
THEORETICAL ORACLE

1. Executive Summary: What We Convey Through the First 70%

Modern optimizing compilers such as LLVM/Clang rely on dozens of discrete transformation passes (e.g., dead code elimination, constant propagation, loop unrolling, instruction combining). Conventionally, software engineers invoke high-level compiler flags such as `-O2` (optimizing for runtime speed) or `-Oz` (optimizing for minimal code size).

The Core Problem (Phase Ordering): Compilers use a single, hardcoded, one-size-fits-all pass sequence for every program. However, code transformations interact non-linearly: an optimization pass can create or destroy opportunities for subsequent passes. Consequently, a pass sequence that shrinks matrix multiplication can severely *bloat* an image convolution or a state-machine routine. Searching the entire space of pass permutations at compile time is computationally intractable (NP-hard).

The Core Project Thesis (70% Deliverable): By inspecting the static structural "DNA" of a program's LLVM Intermediate Representation (IR)—measuring instruction mix, control-flow branching, memory operations, and loop characteristics—a lightweight Machine Learning classifier can predict a custom, short optimization pass sequence tailored to that specific program. This predicted sequence outperforms LLVM's official `-O2` and `-Oz` flags in code size under a strictly constrained compilation budget, with near-zero inference latency (17–45 ms).

2. Benefits of Our Approach Compared to Prior / Traditional Methods

Evaluation Dimension	Traditional Heuristics (<code>-O2</code> / <code>-Oz</code>)	Search-Based Autotuning (Genetic / Iterative)	Our Approach (Static ML Pass Selection)
Adaptability to Program Structure	None (Static, fixed pipeline for all C code)	High (Explores program-specific space)	High (Predicts sequence conditioned on IR features)
Compilation Overhead	Low (Single pass pipeline)	Extremely High (Compiles 500–5000+ times per file)	Ultra-Low (Single inference in ~20 ms + single compile)
Code Size vs <code>-O2</code> / <code>-Oz</code>	Baseline (0.0% by definition)	+10% to +15% reduction (after hours of search)	+10.36% average reduction (instantaneous)
Feasibility in Production CI/CD	High	Zero (Prohibitive build times)	High (Integrated directly into build scripts)
Model Interpretability	N/A (Hand-crafted C++ heuristics)	Black-box search history	High (Feature importance ranks IR branch/memory ratios)
Hardware Requirement	CPU	Massive compute clusters	Standard CPU (Trains in <1s on Random Forest)

Key Architectural Advantages Demonstrated:

- Zero Runtime Search Penalty:** Unlike genetic algorithms that require re-compiling a program hundreds of times, our model makes a deterministic classification in under 45 milliseconds.
- Prevention of Destructive Optimizations:** Standard `-O2/-Oz` flags frequently apply loop vectorization and unrolling heuristics that duplicate instructions. Our ML model identifies small loop bodies and selects minimalist simplification pipelines (e.g., `simplifcfg`, `instcombine`), eliminating 45.6% of binary bloat.

3. **Guaranteed Leakage-Free Evaluation:** Unlike naive ML benchmarks that split rows randomly, we enforce strict program-level partitioning: training, validation, and testing share zero source files.

3. Practical Demonstrations: Step-by-Step System Walkthrough

The system is fully implemented and executable. The table below details what is demonstrated at each stage of the pipeline, the exact commands executed, and the physical outputs verified.

Step 1: Environment & Toolchain Diagnostics

```
python scripts/check_environment.py
```

- **What the Demonstration Shows:** Verifies Python 3.12, scikit-learn, pandas, numpy, and autodetects the LLVM toolchain (Clang 23.1.2 and llvmsize). Generates `experiments/environment_manifest.json`.
- **Key Technical Innovation:** Handles modern LLVM 23 architectures that omit standalone `opt.exe` by orchestrating optimization passes through Clang's internal pass manager and cross-platform header resolvers.

Step 2: Benchmark Suite & Zero-Leakage Dataset Splitting

```
cat benchmarks/metadata/manifest.json
```

- **What the Demonstration Shows:** 40 diverse C benchmarks across 9 distinct algorithmic categories: *arrays_loops, sorting, searching, recursion, arithmetic, bitwise_logic, string_processing, memory_structures, and control_flow*.
- **Partitioning:** 27 Train (67.5%), 6 Validation (15%), 7 Held-Out Test (17.5%). Programs are partitioned at the file level so test code is completely unseen.

Step 3: Automated LLVM IR Generation

```
python scripts/build_ir.py
```

- **What the Demonstration Shows:** Compiles all 40 C files into clean LLVM Intermediate Representation (`ir/*.ll`).
- **Critical Implementation Requirement:** Applies `-Xclang -disable-00-optnone`. Without this flag, Clang tags functions with `optnone`, which silently prevents subsequent optimization passes from executing.

Step 4: Deterministic 30-Feature IR Extraction

```
python scripts/extract_features.py
```

- **What the Demonstration Shows:** Parses the LLVM IR grammar and extracts 30 static numerical features across:
 - **Program Size:** Instruction count, basic block count, function count.
 - **Control Flow:** Unconditional/conditional branches, switches, branch-to-instruction ratio.
 - **Memory:** Load, store, alloca, getelementptr (GEP) counts and load/instruction ratios.
 - **Computation:** Integer/floating-point arithmetic, bitwise operators, comparisons.
 - **SSA & Normalization:** PHI nodes, select instructions, instructions per basic block.
- **Output:** Generates `features/ir_features.csv` (40 rows × 30 deterministic feature columns).

Step 5: Candidate Pass Sequences & Limited-Budget Profiling

```
python scripts/run_experiments.py --split all --budget 30
```

- **What the Demonstration Shows:** Evaluates 42 candidate pipelines (Baselines O2, OZ, and candidate sequences S01–S40) using safe optimization passes (instcombine, simplifcfg, sroa, dce, gvn, sccp, licm, mem2reg).
- **Budget Constraint:** Limits sequence evaluations to at most 30 per program to avoid combinatorial explosion. Evaluates 1,280 compilations using 12 parallel threads with thread-safe disk caching in `experiments/results_cache.json`.

Step 6: Supervised Model Training & Feature Importance

```
python scripts/train_model.py --model all
```

- **What the Demonstration Shows:** Merges features and empirical winning labels into dataset/dataset_train.csv and dataset_test.csv. Trains both a **Random Forest Classifier** and a **HistGradientBoosting Classifier**.
- **Results:** Random Forest achieves 100% training accuracy and 57.1% top-3 test accuracy. Models are serialized to models/random_forest_pass_predictor.joblib.

Step 7: Live Single-Program Inference Demonstration

```
python scripts/predict.py --source benchmarks/test/PG005_conv2d.c
```

- **Live Output Displayed:**

```
===== LLVM ML OPTIMIZATION PREDICTION
PIPELINE: PG005_conv2d.c ===== Step 1:
Generated LLVM IR -> (1067.3 ms) Step 2: Extracted 30 static IR features (0.7 ms) Instructions: 139,
Blocks: 24, Branches: 24, Loads/Stores: 32/20 Step 3: ML Model Inference (17.60 ms) Predicted Best
Sequence : S02 (simplifcfg,instcombine)
===== ML Code Size : 346 bytes LLVM -O2
Baseline : 636 bytes LLVM -Oz Baseline : 636 bytes -----
----- ML vs LLVM -O2 : +45.60% size reduction (+290 bytes) ML vs LLVM -Oz : +45.60% size
reduction (+290 bytes) Outcome Verdict : WIN (ML produced smaller code than BOTH!)
=====
```

Step 8: Full Test Suite Evaluation (Held-Out Benchmark Table)

```
python scripts/evaluate.py
```

Program ID	Benchmark Name	Category	-O2 (B)	-Oz (B)	ML Predicted (B)	Oracle (B)	Predicted Pass	Reduction vs -O2	Verdict
PG005	conv2d	arrays_loops	636	636	346	346	S02	+45.60%	WIN
PG010	quicksort	sorting	315	315	315	315	S16	0.00%	TIED
PG015	tree_traverse	recursion	102	102	102	102	S05	0.00%	TIED
PG020	horner_eval	arithmetic	256	256	187	144	S04	+26.95%	WIN
PG026	str_search	strings	145	145	145	145	S05	0.00%	TIED
PG035	decision_tree	control_flow	160	160	160	160	S04	0.00%	TIED
PG040	knapsack	mixed_comp	294	294	294	294	S02	0.00%	TIED

4. Checklist of Items to Present During Evaluation / Viva

1. **Live Terminal Execution:** Run `python scripts/predict.py --source benchmarks/test/PG005_conv2d.c` in the integrated terminal to demonstrate live, un-faked compiler invocation.
2. **Interactive Web Dashboard:** Open `results/dashboard.html` in the web browser. Show the bar chart contrasting ML vs -O2/-Oz, the donut breakdown, and the metric cards.
3. **Research Visualizations:** Open the images in `results/plots/`:
 - `per_program_comparison.png` — Per-benchmark comparative bar chart.
 - `feature_importance.png` — Top features utilized by the model (e.g., branch frequency, memory load ratio).
 - `ml_wins_breakdown.png` — 100% win/tie proportion with zero regressions.
4. **Automated Unit Test Suite:** Run `python scripts/run_tests.py` to show all 5 unit tests passing (verifying zero data leakage, feature extraction validity, and candidate syntax).
5. **Cloud CI/CD & Automated Regression Testing:** Show the automated GitHub Actions pipeline executing the end-to-end suite with green passing checks on remote runners.

5. Roadmap for the Remaining 30% Milestone

- **Pass Pipeline Expansion:** Expand short pipelines (2–4 passes) to deeper sequences (5–8 passes) with loop-invariant code motion (LICM) and scalar replacement of aggregates (SROA).
- **Multi-Objective Pareto Optimization:** Optimize simultaneously for binary size (.text bytes) and runtime execution latency (cycle count via cycle timers).
- **Dynamic Feature Integration:** Combine static IR opcode counts with basic-block execution frequency profiles collected from sample execution traces.